

GPU Sharing and Coordinated Scheduling for Collaborative LLM Inference

Weizhen Zhou
Qingyang Guo
Yihao Huang

Collaborative LLM Inference on multiple home Devices

- A single device may not have enough resources to run a large model
- Multiple devices can work together as a cluster
- Our project builds on **Distributed Llama [1]**

[1] Available: <https://github.com/b4rtaz/distributed-llama>

Tensor and Pipeline Parallelism

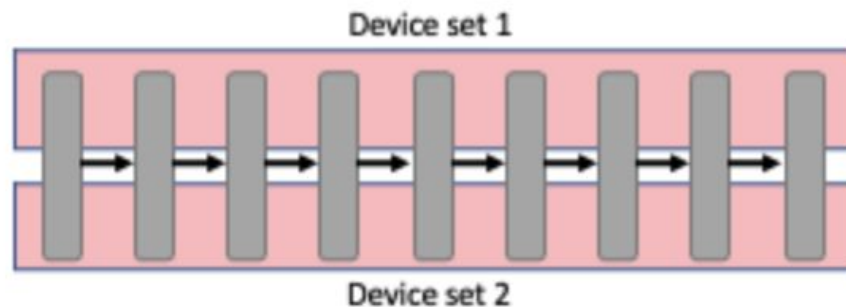
Tensor Parallelism (TP)

- Used by Distributed Llama [1]
- Splits computation within each model layer across devices
- Requires devices to exchange and synchronize partial results

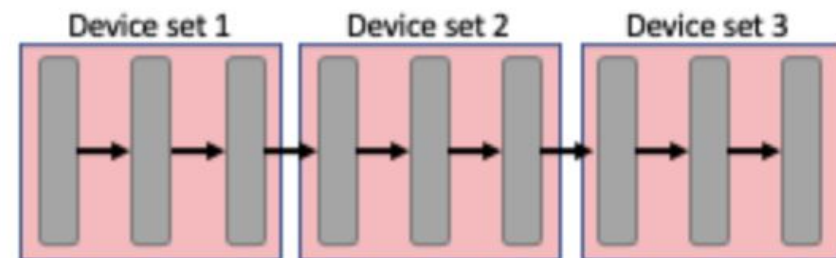
Pipeline Parallelism (PP)

- Places different groups of layers on different devices
- Passes intermediate results from one stage to the next

Tensor Parallelism (TP)



Pipeline Parallelism (PP)



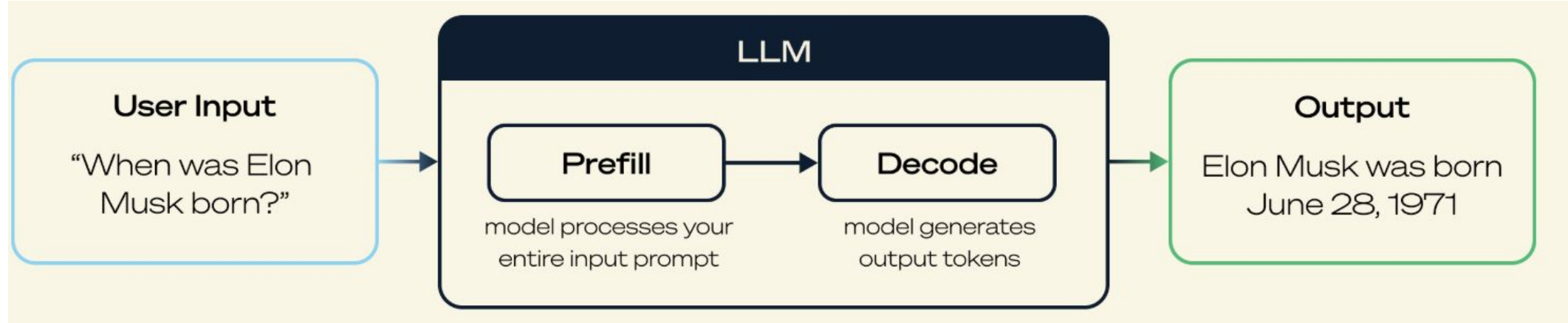
Two Phases of LLM Inference

Prefill

- Processes the entire input prompt
- Builds the KV cache
- Longer prompts require more prefill work

Decode

- Generates output tokens one at a time
- Reuses and extends the KV cache
- Continues until the response is complete



Potential Problem of distributed-llama

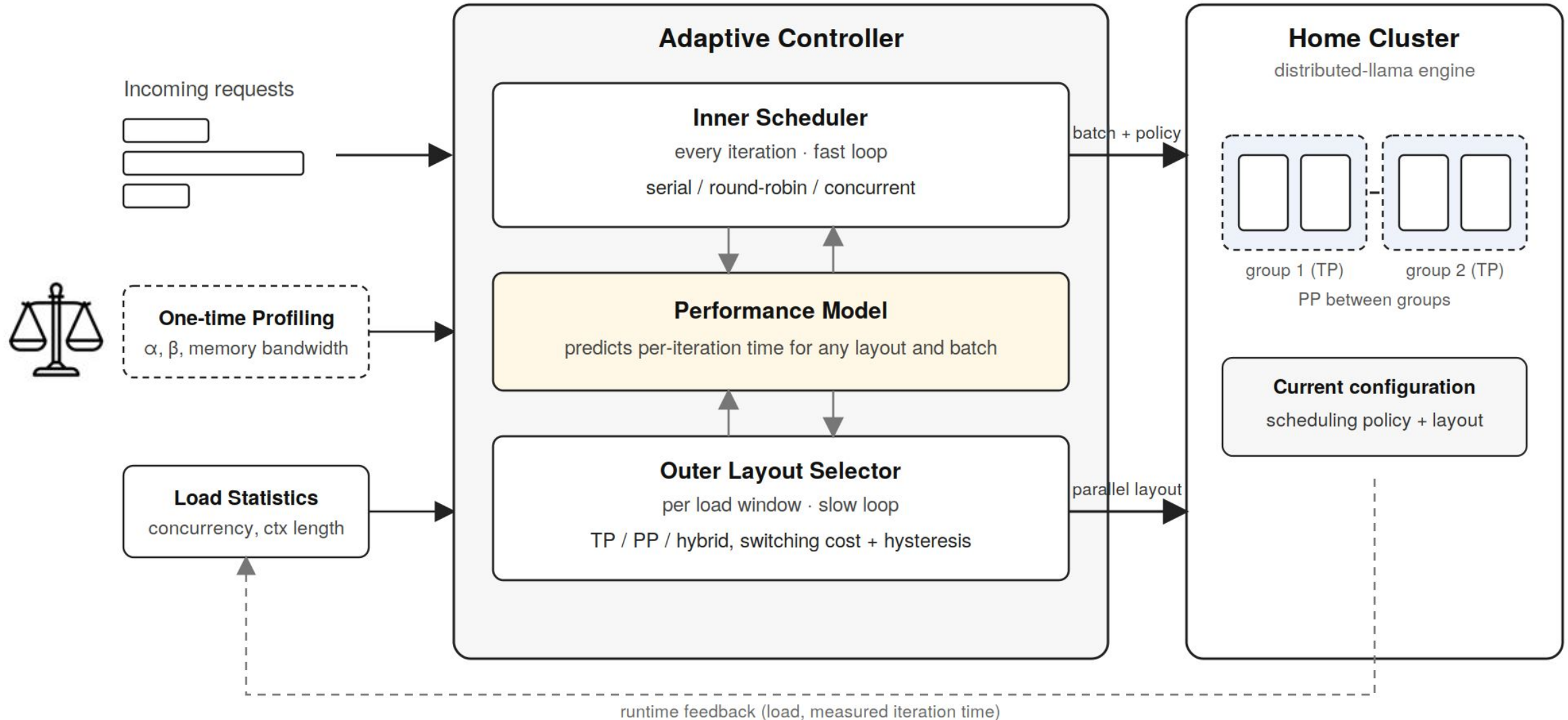
Distributed llama fix scheduling strategy as serial scheduling and parallel strategy as tensor parallelism. And doesn't consider different workload scenarios. Which could lead to:

1. Head-of-line blocking.
2. Tensor parallelism has heavy communication overhead.
3. TP's fixed per-iteration communication cost unamortized.
4. Pipeline parallelism could be more efficient with lots of active request.

Project Structure

1. Conduct wholesome measurement to verify the following assumptions
 - a. Head-of-line blocking.
 - b. TP's heavy communication overhead.
 - c. TP's fixed per-iteration communication cost unamortized.
 - d. PP's gain with lots of active request.
2. Implement a two-layer adaptive controller for dynamic selection of scheduling strategy and parallelism strategy.

Adaptive controller



Stoppers and Plan B

1. Cost of TP and PP exchange.
2. Implementation of PP and controller model.
3. Interconnect speed in real testing environment.
4. Simulation workload design.

Experimental setup

Models

- Llama 3.1 8B
- Llama 3.3 70B
- Q40, Vulkan

Prompts

- ShareGPT (short)
- LongBench (long)

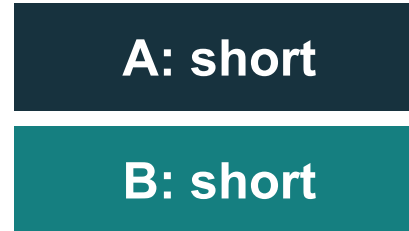
Hardware

- HiPerGator L4
- 2 / 4 / 8 GPUs

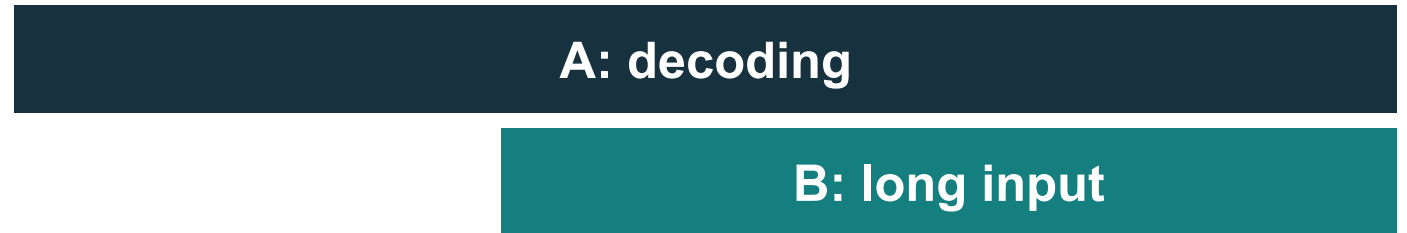
Metrics: Time to First Token (TTFT), Time per Output Token (TPOT), Time to Last Token (TTLT), P99 latency, throughput, slowdown, and fairness

Three arrival scenarios

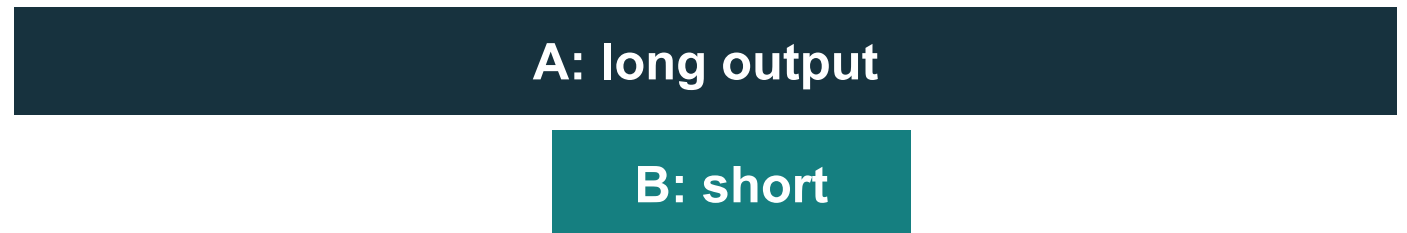
Two short, together



Long input during decode



Short input during long output



time

Three scheduling policies

Serial (FIFO)

One request finishes before the next starts (baseline)



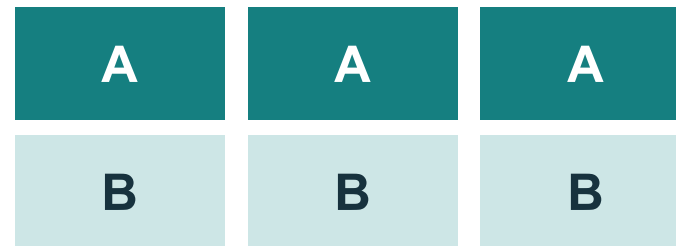
Round-robin

Requests take turns every steps



Concurrent

Requests share each forward step (batching)



Each block is one forward step

time →

Q&A